Sorting

Sean

October 11, 2026



Outline

- Why sorting helps
- Using `sort`
- Custom ordering
- Stability and equal keys
- Common sorting patterns



Introduction

- Sorting puts data into a useful order
- Once data is ordered, many repeated comparisons become unnecessary
- In competitive programming, sorting is often the first step that turns a brute-force idea into an efficient one

1 Why Sorting Helps

2 Using `sort`

3 Custom Ordering

4 Common Sorting Patterns



Why Sorting Helps



Why Sorting Helps

- Sorting groups related values together
- It reveals smallest, largest, nearest, and duplicate values
- It lets us process elements in a meaningful order
- It often changes a problem from “compare everything with everything” to “scan once after sorting”



Sorting Complexity

- C++ `sort` uses $\mathcal{O}(n \log n)$ comparisons
- For integers, this is usually fast enough for $n \leq 2 \times 10^5$
- If one comparison costs $\mathcal{O}(C)$, sorting costs $\mathcal{O}(Cn \log n)$; comparing long strings may cost more than comparing integers

$$\mathcal{O}(n^2) \quad \Longrightarrow \quad \mathcal{O}(n \log n + n)$$



Simple Sorts

- Bubble sort, selection sort, and insertion sort are common beginner sorting algorithms
- They usually take $\mathcal{O}(n^2)$ time
- That is fine for small inputs, but too slow for many CP constraints
- In competitive programming, we usually use `sort` unless a problem needs something more specialized



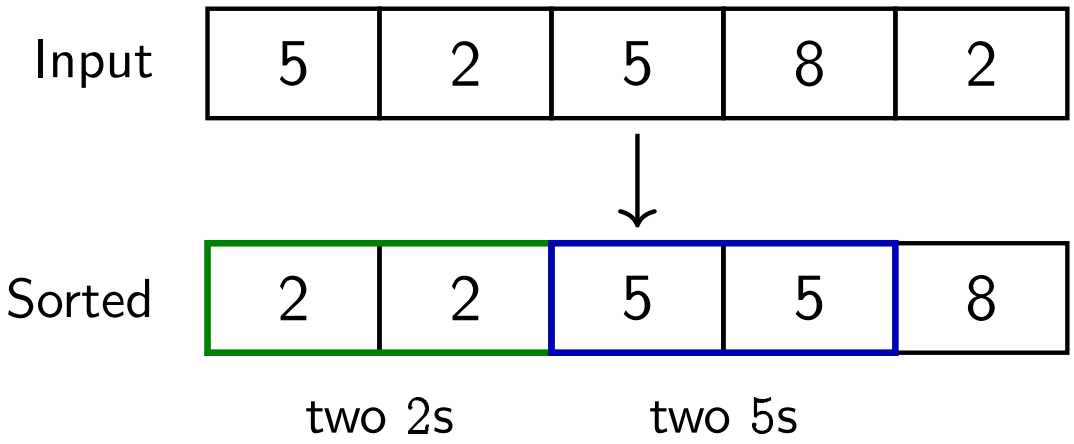
Example: Duplicates

Task

Given n numbers, decide whether any value appears at least twice.

- Brute force: compare every pair, $\mathcal{O}(n^2)$
- After sorting, equal values become adjacent
- Scan neighboring pairs only, $\mathcal{O}(n \log n)$ total

Trace: sorting reveals duplicates



After sorting, every copy of a value lies in one consecutive group.
Compare adjacent elements, or scan whole groups to count frequencies.

1 Why Sorting Helps

2 Using `sort`

3 Custom Ordering

4 Common Sorting Patterns



Using `sort`



Sorting a Vector

```cpp
vector<int> a = {5, 1, 4, 1, 3};
sort(a.begin(), a.end()); // a = {1, 1, 3, 4, 5}
for (int x : a) cout << x << ' '; // 1 1 3 4 5
cout << is_sorted(a.begin(), a.end()); // 1: true
vector<int> empty;
sort(empty.begin(), empty.end()); // safe; no elements
vector<int> one = {7};
sort(one.begin(), one.end()); // still {7}
// Range is [begin(), end()); end() is not an element.
// Sorting changes the order, not the size or values.
```

C++17; snippets assume `using namespace std;` and headers `<algorithm>`, `<vector>`, `<iostream>`, `<functional>`, `<utility>`, `<string>`, and `<array>`.



Decreasing Order

```cpp
vector<int> a = {5, 1, 4, 1, 3};
sort(a.begin(), a.end(), greater<int>());
for (int x : a) cout << x << ' '; // 5 4 3 1 1
cout << is_sorted(a.begin(), a.end(), greater<int>());
// 1: is_sorted must use the same ordering rule.
vector<int> b = {5, 1, 4, 1, 3};
sort(b.rbegin(), b.rend()); // b = {5, 4, 3, 1, 1}
// Reverse iterators give another descending-sort idiom.
// reverse(b.begin(), b.end()) only reverses the order;
// it does not sort arbitrary unsorted input.
```



Sorting Part of an Array

```cpp
vector<int> a = {9, 5, 4, 3, 8};
int l = 1, r = 3; // inclusive indices [1, 3]
sort(a.begin() + l, a.begin() + r + 1);
// a = {9, 3, 4, 5, 8}; outside elements stay in place.
for (int x : a) cout << x << ' '; // 9 3 4 5 8
sort(a.begin() + 2, a.begin() + 2); // empty range
// For [l, r], require 0 <= l <= r < a.size().
// For [l, r), use begin()+l and begin()+r instead.
// Check the indices BEFORE forming the iterators.
```



Arrays and strings in code

```
1  int raw[] = {3, 1, 2};
2  sort(raw, raw + 3); // raw = {1, 2, 3}; pointer range
3  array<int, 4> a = {7, 2, 7, 5};
4  sort(a.begin(), a.end()); // a = {2, 5, 7, 7}
5  string s = "cabba";
6  sort(s.begin(), s.end()); // s = "aabbc"
7  cout << s; // aabbc: characters, not whole words
8  // sort needs random-access iterators, as above.
9  // list has .sort(); set/map maintain their own order.
```



1 Why Sorting Helps

2 Using `sort`

3 Custom Ordering

4 Common Sorting Patterns



Custom Ordering



Sorting Pairs

```cpp
vector<pair<int, int>> points = {
    {2, 9}, {1, 7}, {2, 3}, {1, 4}};
sort(points.begin(), points.end());
// points = {{1, 4}, {1, 7}, {2, 3}, {2, 9}}
for (auto [x, y] : points)
  cout << x << ',' << y << ' '; // 1,4 1,7 2,3 2,9
// Lexicographic: first component, then second on ties.
// Sorting by first ONLY requires a custom comparator.
```



Custom Comparator

```
1  struct Item { int score; string name; };
2  vector<Item> v = {{70,"Bob"},{90,"Zoe"},{70,"Ana"}};
3  sort(v.begin(), v.end(), [](const Item& a,
4                              const Item& b) {
5    return a.score > b.score; // true: a goes before b
6  });
7  for (const Item& x : v) cout << x.score << ' ';
8  // 90 70 70: descending scores
9  // Bob and Ana are equivalent under this comparator.
10 // sort does NOT promise their relative order.
```



Tie-Breaking

```
struct Item { int score; string name; };
vector<Item> v = {{70,"Bob"},{90,"Zoe"},{70,"Ana"}};
sort(v.begin(), v.end(), [](const Item& a,
                            const Item& b) {
  if (a.score != b.score) return a.score > b.score;
  return a.name < b.name;
});
for (const Item& x : v) cout << x.name << ' ';
// Zoe Ana Bob: score descending, name ascending.
// Do not negate a key or subtract keys just to compare;
// direct comparisons avoid integer overflow.
```



Capturing Outside Data

```
1  const vector<int> pos = {2, 0, 1}; // ID -> priority
2  vector<int> v = {0, 1, 2};
3  sort(v.begin(), v.end(), [&](int a, int b) {
4    return pos[a] < pos[b]; // lower priority first
5  });
6  for (int id : v) cout << id << ' '; // 1 2 0
7  // [&] captures outside variables by reference.
8  // Every ID must be a valid index in pos.
9  // Keep pos fixed throughout the sort.
```



A strict weak order

- `cmp(x,x)` must be false: an element never comes before itself
- If a comes before b and b before c, then a must come before c
- If neither element comes before the other, they are equivalent under the rule; these ties must also be transitive
- Comparing one fixed key, or checking keys in a fixed priority order, gives a consistent rule



Why <= breaks a comparator

```
auto bad = [](int a, int b) { return a <= b; };
cout << bad(5, 5); // 1: says 5 comes before itself
// Never pass bad to sort: it violates the requirements.
auto good = [](int a, int b) { return a < b; };
cout << good(5, 5); // 0: correct for equivalent values
cout << good(2, 5); // 1: 2 comes before 5
cout << good(5, 2); // 0: the reverse must be false
// return a - b is wrong: both nonzero signs become
↪  true.
// The subtraction itself can overflow too.
```

Comparator Mistakes

- Using <=, >=, or a subtraction result as the comparator
- Changing elements or outside state during comparisons
- Assuming `sort` preserves the input order of equivalent elements
- Violating the comparator requirements causes undefined behavior, including possible crashes or wrong results



Stability and equal keys

Sort by decreasing score; Bob and Ana both have score 70.

Rule	**Order of names**
Input	Bob, Zoe, Ana
`sort` by score	Zoe, then Bob/Ana in either order
`stable_sort` by score	Zoe, Bob, Ana
`sort` by score, then name	Zoe, Ana, Bob

`stable_sort` preserves input order among equivalent elements. Equivalence depends on the comparator, not on `operator==`.



stable_sort in code

```
1  struct Item { int score; string name; };
2  vector<Item> v = {{70,"Bob"},{90,"Zoe"},{70,"Ana"}};
3  stable_sort(v.begin(), v.end(), [](const Item& a,
4                                     const Item& b) {
5    return a.score > b.score; // compare the score only
6  });
7  for (const Item& x : v) cout << x.name << ' ';
8  // Zoe Bob Ana: Bob stays before Ana on score ties.
9  // Use this when input order should break ties.
```

C++17: $\mathcal{O}(n \log n)$ comparisons with enough extra memory; $\mathcal{O}(n \log^2 n)$ otherwise. Use `sort` when stability is unnecessary.



1 Why Sorting Helps

2 Using `sort`

3 Custom Ordering

4 Common Sorting Patterns



Common Sorting Patterns



Pattern: Sort Then Scan

- Sort the data
- Scan from left to right
- Keep only the information needed from previous elements
- This often gives $\mathcal{O}(n \log n)$ time

Detecting and removing duplicates

```cpp
vector<int> a = {5, 2, 5, 8, 2};
sort(a.begin(), a.end()); // {2, 2, 5, 5, 8}
cout << (adjacent_find(a.begin(), a.end()) != a.end());
// 1: at least one equal neighboring pair exists
auto new_end = unique(a.begin(), a.end());
cout << a.size(); // 5: size has NOT changed
cout << new_end - a.begin(); // 3: logical unique length
a.erase(new_end, a.end()); // a = {2, 5, 8}
for (int x : a) cout << x << ' '; // 2 5 8
// Idiom: a.erase(unique(a.begin(),a.end()),a.end());
// unique removes only consecutive duplicates: sort
    ↪ first.
// Empty ranges are safe; keep copies if counts matter.
```



Scanning equal-value groups

```cpp
vector<int> a = {5, 2, 5, 8, 2};
sort(a.begin(), a.end()); // {2, 2, 5, 5, 8}
int n = (int)a.size();
for (int i = 0; i < n; ) {
  int j = i + 1;
  while (j < n && a[j] == a[i]) ++j;
  cout << a[i] << ':' << j - i << ' '; // value:count
  i = j; // next unprocessed group
}
// Output: 2:2 5:2 8:1
// Each index is visited once during the scan: O(n).
// Total time including sorting: O(n log n).
```



Keeping original positions

```cpp
vector<int> a = {50, 20, 50};
vector<pair<int, int>> order; // {value, original index}
for (int i = 0; i < (int)a.size(); ++i)
  order.push_back({a[i], i});
sort(order.begin(), order.end());
// order = {{20, 1}, {50, 0}, {50, 2}}
for (auto [value, index] : order)
  cout << index + 1 << ' '; // 2 1 3: one-based
// Pair order breaks value ties by original index.
// a stays unchanged; only order is rearranged.
// Store positions before sorting if output needs them.
```



Example: Apartments

CSES: Apartments

Applicants have desired apartment sizes. Apartments have actual sizes. An applicant accepts an apartment if the size differs by at most k. Maximize the number of matches.

- Sort applicants and apartments by size
- Too small for the current applicant: discard that apartment
- Too large: discard that applicant; later apartments are even larger
- Otherwise match them and advance both pointers



Trace: Apartments

Sorted desired sizes $[45, 60, 60, 80]$, apartments $[30, 60, 75]$, $k = 5$.

Desired	Apartment	Action	Matches
45	30	Too small: next apartment	0
45	60	Too large: next applicant	0
60	60	Match: advance both	1
60	75	Too large: next applicant	1
80	75	Match: advance both	2

An optimal matching can include this smallest compatible pair: swap assignments without losing a match.



Apartments in code

```cpp
vector<long long> a = {60,45,80,60}, b = {30,60,75};
long long k = 5;
sort(a.begin(), a.end());
sort(b.begin(), b.end());
int i = 0, j = 0, matches = 0;
while (i < (int)a.size() && j < (int)b.size()) {
  if (b[j] < a[i] - k) ++j;       // too small
  else if (b[j] > a[i] + k) ++i; // too large
  else { ++matches; ++i; ++j; }  // match: use both
}
cout << matches; // 2
// long long keeps a[i] +/- k safe for these inputs.
// Time: O(n log n + m log m + n + m).
```



Pattern: Sort Events

- Convert each important moment into an event
- Sort events by position or time
- Decide how ties are handled before scanning
- Process events in order while maintaining the current state
- This is the beginning of sweep-line thinking



Example: Restaurant Customers

CSES: Restaurant Customers

Given arrival and leaving times of customers, find the maximum number of customers present at the same time.

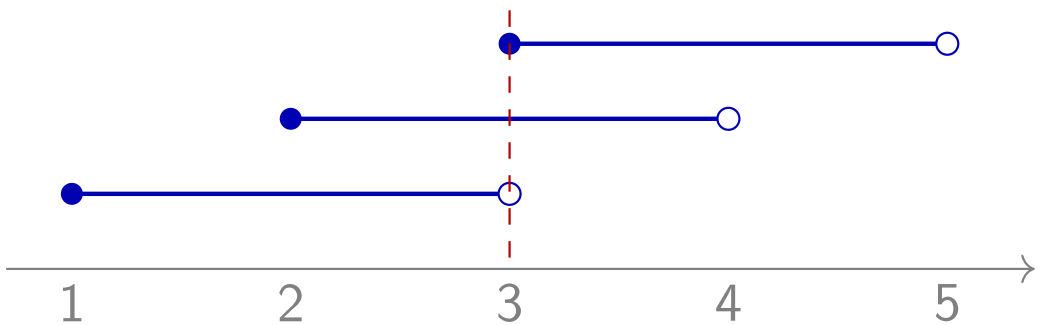
- Create a $+1$ event for each arrival
- Create a -1 event for each leaving time
- CSES guarantees all arrival and leaving times are distinct
- Sort events by time and scan them in order



Event ties depend on interval meaning

For problems that allow equal event times, choose the endpoint rule first.

Intervals	**At a tied time**	**Maximum below**
Half-open $[l, r)$	Departures before arrivals	2
Closed $[l, r]$	Arrivals before departures	3



Shown: half-open intervals $[1, 3)$, $[2, 4)$, $[3, 5)$. If the right endpoints were included, all three would contain time 3.

Restaurant Customers in code

```cpp
vector<pair<int,int>> visits = {{5,8},{2,4},{3,9}};
vector<pair<int, int>> events; // {time, change}
for (auto [arrival, leaving] : visits) {
  events.push_back({arrival, +1});
  events.push_back({leaving, -1});
}
sort(events.begin(), events.end()); // time, then change
int current = 0, best = 0;
for (auto [time, change] : events) {
  current += change;
  best = max(best, current);
}
cout << best; // 2; sorting 2*n events: O(n log n)
```



Ties: -1 before $+1$ models half-open intervals $[l, r)$.

Trace: tied half-open events

Extension with ties: $[1,3)$, $[2,4)$, $[3,5)$.

Time	Change	Current	Best
1	$+1$	1	1
2	$+1$	2	2
3	-1	1	2
3	$+1$	2	2
4	-1	1	2
5	-1	0	2

At time 3, the departing customer leaves before the new one arrives. Processing $+1$ first would incorrectly count 3 customers here.



What Sorting Does Not Solve

- Sorting changes the order of elements
- If original positions matter, store the index before sorting
- If the data changes after every query, sorting once may not be enough
- Sorting is a tool, not a proof of correctness by itself



Choosing a sorting operation

Need	**Use**
Increasing order	`sort(begin, end)`
Decreasing order	`greater<T>()` or reverse iterators
Order by several keys	Strict comparator with explicit tie rules
Keep input order on ties	`stable_sort` with that comparator
Distinct values	Sort, then `unique` and `erase`
Count copies of each value	Sort, then scan equal-value groups
Recover original positions	Store `{value, index}` before sorting



Takeaways

- `sort` uses $\mathcal{O}(n \log n)$ comparisons
- Use sorting to reveal order, adjacency, duplicates, and extremes
- Use strict comparators; choose tie-breaking or stability deliberately
- Many CP solutions are: sort, then scan carefully



The End

Thanks for listening.

© 2026 Sean. Please do not repost or redistribute without permission.

